

2026-09-30

Refactoring and rework of dabestr

Yishan Mai 

Duke-NUS Medical School, Singapore

This template was last revised on 2026-03-24.

Executive Summary

dabestr is the principal R implementation of estimation statistics, a data-analysis framework that reports effect sizes and confidence intervals in place of null-hypothesis significance testing. It is on CRAN and developed openly on GitHub, and implements methods described in two papers from our group (Ho et al. 2019; Lu et al. 2026). It has grown from two-group mean differences into a package spanning repeated measures, delta–delta (2×2) designs, mini-meta analysis, and binary proportions.

That growth has outpaced the original architecture. First-class statistical concepts are encoded implicitly across the arguments of one entry-point function, a set of design flags, and a heterogeneous result object that the plotting layer inspects directly. The package works, but is increasingly hard to extend, test, and integrate with the R ecosystem.

Proposal. A focused modernization that makes the existing statistical grammar explicit: a documented, standardized, composable estimation-result object (built on S7), computation separated from visualization, the current implementation frozen as a regression target, and full backward compatibility. No new statistical methodology is proposed; consolidating the engineering keeps scientific risk low and yields infrastructure other R packages can build on.

Deliverables (6 milestones, 9 months). (1) architecture specification and result-object schema; (2) frozen regression corpus in CI; (3) standardized result object with tidy accessors; (4) plots refactored onto the public contract; (5) backward-compatibility layer, deprecation path, and simplified delta–delta interface; (6) documentation, contributor guide, and CRAN release. Budget: USD \$10,000, all developer labor, allocated by milestone and paid on the ISC’s standard 50%/50% schedule.

Signatories

Project team

dabestr is developed by the Claridge-Chang lab (ACCLAB) at Duke-NUS Medical School and collaborators, who authored the methods it implements and also maintain DABEST-Python and estimationstats.com. One primary developer (about 0.4 FTE over 9 months) will come from the existing maintainer team, with design review and statistical validation from the co-applicants at no cost to the grant.

Track record: github.com/ACCLAB/dabestr (CRAN), github.com/ACCLAB/DABEST-Python, estimationstats.com, and the two papers in References (Ho et al, 2019; Lu et al., 2026).

Contributors

The authors of this proposal will all be on the core project team.

Consulted

Not currently applicable, but the Milestone 1 design will be opened for public comment before implementation.

The Problem

Estimation statistics has become a mainstream alternative to significance testing across the life and behavioural sciences, reinforced by journal policy changes and a growing tool ecosystem. dabestr is the main way R users produce estimation analyses and plots. But the package exposes no stable, documented representation of an estimation result. A completed analysis is a large list carrying, simultaneously, raw data, comparison indices, control/test summaries, bootstrap and permutation results, design flags, and plotting configuration (labels, limits, colour mappings). The same object serves as analysis specification, results container, data carrier, design record, and plot configuration all at once.

Two structural consequences follow:

1. Data ingestion, experimental design, and analysis configuration are collapsed into one entry point. The `load()` function now encodes grouping structure, comparison structure (`idx`), paired/repeated-measures design, subject identifiers, proportional outcomes, mini-meta analysis, and delta-delta analysis. Each new method has historically been added as another argument, flag, or specialized code path rather than as an explicit component of a shared model.

2. Visualization is coupled to internal statistical state. The plotting layer branches on whether an analysis is delta-delta or mini-meta, on the number and structure of comparison groups, and on other design-specific fields, then assembles the panels accordingly. Presentation logic therefore depends on knowledge of the internal object rather than on a documented contract.

Why is it a problem, and who does it affect?

Users cannot reliably program against results — extracting effect sizes, intervals, or bootstrap distributions for downstream reporting means reaching into undocumented internal structure that can change between releases.

The maintainers face rising per-feature cost. Our own roadmap illustrates this directly: delta-delta alone has accumulated three separate open requests (simplify its input interface; support a “multiplex”

form; support pooled variance between control and test groups). Each currently implies bespoke input handling and plotting logic. This is the accumulation pattern in miniature.

The wider R ecosystem cannot easily consume estimation results. Because there is no standardized, tidy-friendly result object, alternative or ggplot2-native plotting layers, report-generation tools, and other statistical packages cannot build on dabestr output without depending on its internals.

What will solving it enable?

A documented estimation-result object turns dabestr from a self-contained analysis-and-plot tool into reusable infrastructure: results become ordinary, inspectable, composable R objects; visualization becomes one consumer of a public contract rather than a privileged insider; and future estimands, designs, resampling procedures, and plotting systems can plug into stable interfaces instead of adding specialized pathways. It also enables principled harmonization of concepts with DABEST-Python where that serves users, without forcing non-idiomatic R.

Existing work

dabestr is itself the behavioural reference this project preserves; DABEST-Python and estimation-stats.com implement the same methods for other audiences. General R packages cover pieces of the workflow (e.g. boot and infer for resampling, effectsize and emmeans for effect sizes, ggdist for distribution graphics), but to our knowledge none offers a documented result object for bootstrap-coupled estimation designs that a separate plotting layer can consume.

An example in-text citation ([Wickham 2016](#)).

The proposal

Overview

We separate the entangled layers of the package (data → design → contrast/estimand → resampling/inference → presentation) into explicit, documented components and reconnect them through a stable contract. Scope is architecture and interface, not new methods. A defining advantage is that the correct behaviour already exists: for every supported design and estimand we can freeze current point estimates, bootstrap distributions, confidence intervals, permutation results, and graphical semantics, and require the modernized code to reproduce them. This removes most scientific risk and makes each milestone objectively checkable. The community gains a stable, tidy-friendly interface to estimation results, a lower barrier to building on dabestr, and a faster path to long-requested features.

Detail

Minimum Viable Product

Milestones 1–3: a published architecture specification, the frozen regression corpus, and the standardized S7 result object with tidy accessors for effect sizes, intervals, and resampling distributions, reproducing current outputs.

Architecture

Five explicit layers: data (observations, groups, subject IDs); design (grouping and comparison structure, paired, delta–delta, mini-meta, proportions); contrast/estimand (the effect size and how groups are compared); resampling/inference (bootstrap, permutation, intervals); and presentation (panels, labels, colours). The result object is an S7 class with documented fields and accessors;

plotting reads only this object. The historical `load()` $\rightarrow$ `effectsize()` $\rightarrow$ `dabestplot()` workflow is preserved through a compatibility layer.

Assumptions

current outputs are the correct target, so a frozen corpus defines “unchanged behaviour”; S7 is stable and acceptable to CRAN and users; the existing workflow can be preserved with a warning-based deprecation path; and a maintainer is available for about 0.4 FTE over nine months.

External dependencies

S7; dabestr’s existing plotting and data-handling dependencies; a testing framework and GitHub Actions CI; CRAN review; the nbdev-derived documentation pipeline; DABEST-Python as a conceptual reference only.

Project plan

Start-up phase

The project separates the currently-entangled layers of the package — data $\rightarrow$ design $\rightarrow$ contrast/estimand $\rightarrow$ resampling/inference $\rightarrow$ presentation — into explicit, documented components, then reconnects them through a stable contract. We deliberately scope the work to architecture and interface, not new statistical methods.

dabestr already exists as a full package and Github repository, so the project can be started straight away. A defining advantage is that the correct behaviour already exists. The current package is a rich regression specification: for every supported design and estimand we can freeze current point estimates, bootstrap distributions, confidence intervals, permutation results, and graphical semantics, and require the modernized code to reproduce them. This removes most of the scientific risk and makes each milestone objectively checkable.

Technical delivery

Milestone 1 — Architecture specification and result-object schema (Month 1–2). Deliverable: a public design document (published to the package website and opened as an RFC issue) specifying (a) the separation of design, contrast, estimate, and analysis concepts; (b) the fields, classes, and accessor methods of a standardized result object (implemented with the S7 class system); and (c) the documented contract between analysis output and visualization input. Acceptance: design document merged; community comment period opened; committee-visible.

Milestone 2 — Reference regression corpus (Month 2–3). Deliverable: a frozen corpus of current outputs across all supported designs and estimands (two-group, repeated measures, delta–delta, mini-meta, proportions), covering point estimates, bootstrap distributions, CIs, permutation results, and key graphical semantics, wired into CI. Acceptance: corpus reproducible from a tagged commit; CI green.

Milestone 3 — Standardized result object with decoupled computation (Month 3–5). Deliverable: the estimation-result object implemented, with computation producing it independently of any plotting concern; public accessors for effect sizes, intervals, and resampling distributions returning tidy data. Acceptance: all Milestone-2 regression checks pass; documented accessors covered by tests.

Milestone 4 — Visualization refactored onto the public contract (Month 5–7). Deliverable: the existing plots re-expressed as consumers of the standardized object through the documented contract,

so figures are derived from the result schema rather than from internal state. Acceptance: rendered figures match the frozen graphical corpus; no plotting code reads undocumented internals.

Milestone 5 — Backward-compatibility and deprecation path (Month 6–8). Deliverable: the historical `load()` `|> effect_size()` `|> dabest_plot()` workflow preserved via a compatibility layer over the new architecture, with a documented, warning-based deprecation path for any interface that changes. As a concrete first application of the cleaner design, the delta–delta input interface is simplified. Acceptance: existing user scripts and the current tutorials run unmodified; migration guide published.

Milestone 6 — Documentation, maintenance infrastructure, and CRAN release (Month 8–9). Deliverable: an overhauled documentation front end (a proper introductory abstract for the nbdev-derived docs, which currently begin abruptly), a website changelog page, a migration guide, a contributor guide describing the new architecture, corrected terminology in tutorials (SD vs SEM/BSE), and a CRAN release. Acceptance: package on CRAN; docs and changelog live; contributor guide published.

Other aspects

Everything is developed in the open under Apache-2.0, with issues and pull requests welcome; design documents and the migration guide are published on the package website. We will post an announcement and a delivery post on the R Consortium blog, share quarterly progress, announce through the package’s existing user channels and social media, propagate improvements to `estimationstats.com` (which is built on `dabestr`), and present at `useR!` or a similar venue and at ISC meetings (travel funded outside this grant). Beyond funding, the ISC could help by circulating the Milestone 1 design to reviewers with interests in tidy design, `S7`, and statistical-software architecture, and by advising on emerging community conventions for result objects.

Budget & funding plan

We request USD \$10,000 for developer labor only, with no hardware, cloud, travel, publication, or indirect costs. The project needs roughly 0.4 FTE over nine months from an existing maintainer; the grant funds part of this and the ACCLAB team contributes the remainder.

Funds follow the ISC’s standard schedule: 50% (\$5,000) on signing and 50% (\$5,000) on completion. The per-milestone allocation lets the ISC fund the project in part if needed (e.g. Milestones 1–3, the minimum viable product).

Success

Definition of done

The `S7` result object and accessors are implemented, documented, and tested; all regression checks pass for every supported design and estimand, including rendered figures; plotting reads only the documented object; existing scripts and tutorials run unmodified with a warning-based deprecation path and migration guide; the delta–delta input interface is simplified; and the release is on CRAN with documentation, changelog, and contributor guide live.

Measuring success

Each milestone is reported publicly against its acceptance check. Along the way: the regression corpus stays green in CI; the Milestone 1 design draws community comment that is addressed before implementation; the remaining delta–delta requests (multiplex, pooled variance) are scoped as tractable additions on the new architecture; and, after release, other packages, reports, or plotting layers use the public accessors.

Future work

The architecture is designed to make the following cheaper for the team and community, though none is a deliverable here: new methods on the roadmap (revised permutation algorithm, multiplex and pooled-variance delta–delta, sina plots for large samples); alternative or ggplot2-native plotting layers and reporting tools built on the result object; new estimands, designs, and resampling procedures through stable interfaces; and harmonization with DABEST-Python.

Wickham, Hadley. 2016. *Ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York.
<https://ggplot2.tidyverse.org>.